

Algorithm & problem solving

1.QuickSort

Quick Sort is a divide-and-conquer sorting algorithm that efficiently arranges elements in ascending or descending order. It works by selecting a pivot element, partitioning the array so that smaller elements are placed to the left of the pivot and larger elements to the right, and then recursively applying the same process to the left and right subarrays until the entire array is sorted.

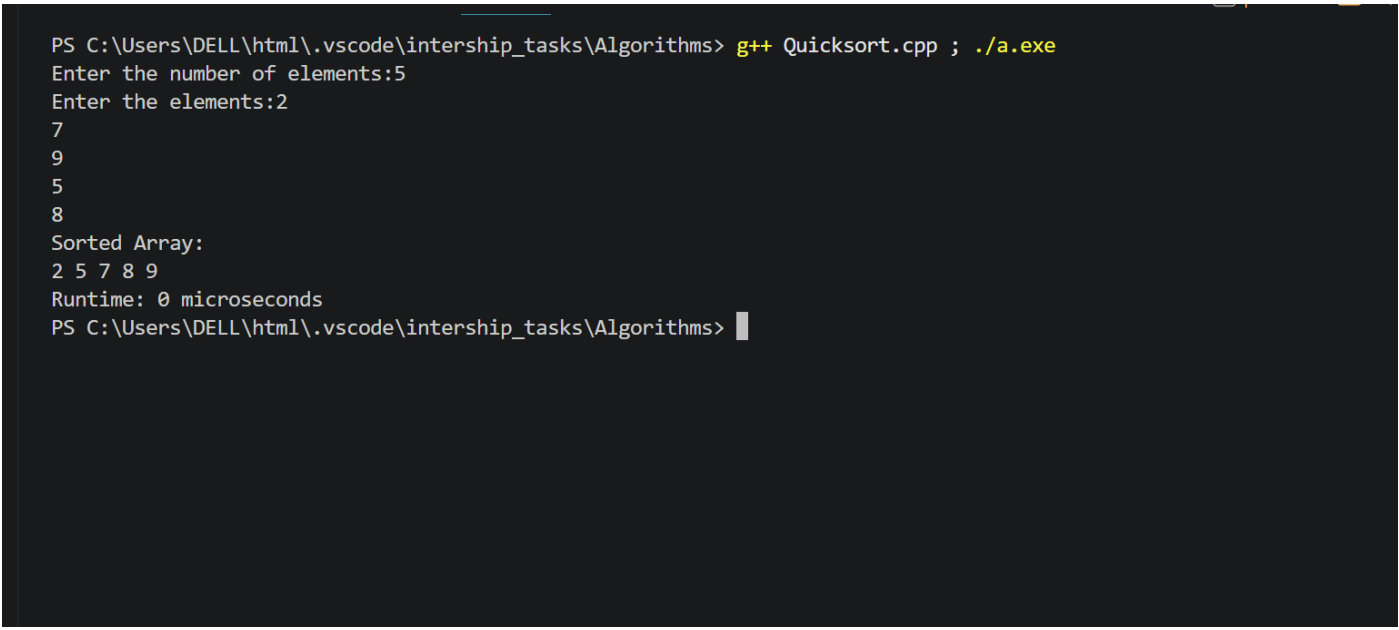
Compile command

```
g++ quicksort.cpp
```

Run command

```
./quicksort.exe
```

Screenshot



```
PS C:\Users\DELL\html\.vscode\internship_tasks\Algorithms> g++ Quicksort.cpp ; ./a.exe
Enter the number of elements:5
Enter the elements:2
7
9
5
8
Sorted Array:
2 5 7 8 9
Runtime: 0 microseconds
PS C:\Users\DELL\html\.vscode\internship_tasks\Algorithms>
```

Sample Input

Number of elements:5

Enter elements:

2 7 9 5 8

Sample output

Sorted array:

2 5 7 8 9

Runtime:0 microseconds

2.Binary Search

Binary Search is an efficient searching algorithm used to find a specific element in a sorted array. It works by repeatedly comparing the target element with the middle element of the array. If the target is smaller than the middle element, the search continues in the left half; otherwise, it continues in the right half. This process repeats until the element is found or the search space becomes empty.

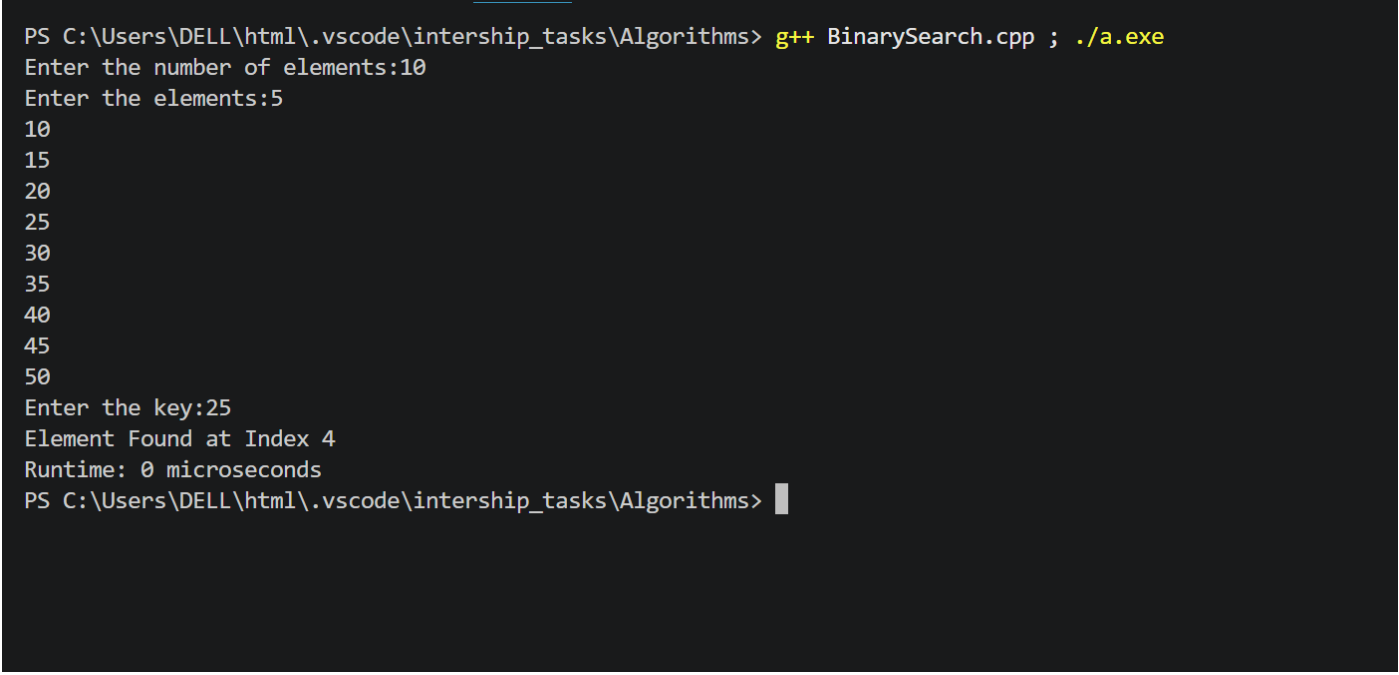
Compile command

```
g++ BinarySearch.cpp
```

Run command

```
./BinarySearch.exe
```

Screenshot



```
PS C:\Users\DELL\html\.vscode\internship_tasks\Algorithms> g++ BinarySearch.cpp ; ./a.exe
Enter the number of elements:10
Enter the elements:5
10
15
20
25
30
35
40
45
50
Enter the key:25
Element Found at Index 4
Runtime: 0 microseconds
PS C:\Users\DELL\html\.vscode\internship_tasks\Algorithms> |
```

Sample Input

Number of elements:10

Elements:

5 10 15 20 25 30 35 40 45 50

Enter the key:25

Sample Output

Element found at index 4

Runtime: 0 microseconds

3.Knapsack

The 0/1 Knapsack Problem is a classic optimization problem in which a knapsack has a fixed weight capacity, and a set of items is given, each with a weight and a value. The objective is to select a combination of items that maximizes the total value without exceeding the knapsack's capacity. Each item can either be included (1) or excluded (0), so items cannot be taken partially.

Compile command

g++ Knapsack.cpp

Run command

./Knapsack.exe

Screenshot

```
PS C:\Users\DELL\html\.vscode\internship_tasks\Algorithms> g++ Knapsack.cpp ; ./a.exe
Enter the number of iteams:3
Enter the weights:10
20
30
Enter the values:60
100
120
Enter the capacity:50
Maximum Profit = 220
Runtime: 0 microseconds
PS C:\Users\DELL\html\.vscode\internship_tasks\Algorithms> |
```

Sample Input

Number of iteams:3

Enter the weigths:

10 20 30

Enter the values:

60 100 120

Enter the capacity:50

Sample Output

Maximum profit:220

Runtime:0 microseconds

